

Patrón de Diseño Observer

1. Problema

En determinados sistemas, un objeto puede cambiar su estado o producir un evento y existen otros objetos que necesitan enterarse de esos cambios automáticamente.

El problema aparece cuando un objeto debe informar a múltiples objetos interesados cada vez que ocurre una modificación. Una solución directa sería que el objeto principal conociera y notificara individualmente a cada uno de ellos. Sin embargo, esto genera un alto nivel de dependencia entre los objetos.

Por ejemplo, podemos considerar un sistema de noticias. Un canal publica una nueva noticia y tiene varios usuarios suscriptos. Cada vez que se publica una noticia, todos los usuarios deberían recibir una notificación.

Si el canal tuviera que conocer directamente a cada usuario, agregar, eliminar o modificar los usuarios podría requerir cambios en la clase principal. Esto haría que el sistema fuera más difícil de mantener y ampliar.

Por lo tanto, el problema que busca resolver Observer es **permitir que un objeto notifique automáticamente a múltiples objetos interesados cuando ocurre un cambio, sin establecer una dependencia fuerte entre ellos.**

Situaciones en las que puede utilizarse

Algunos ejemplos de situaciones donde puede resultar útil son:

- Sistemas de notificaciones.
- Suscripciones a canales de noticias.
- Actualización automática de interfaces gráficas.
- Sistemas de eventos.
- Monitoreo de cambios en datos.
- Aplicaciones donde diferentes componentes necesitan reaccionar ante un mismo evento.

2. Solución

El patrón Observer propone establecer una relación de **uno a muchos** entre objetos.

El objeto que genera el cambio se conoce como **Sujeto (Subject)**, mientras que los objetos que desean recibir información sobre esos cambios se denominan **Observadores (Observers)**.

En el diagrama UML desarrollado para este proyecto, el patrón está compuesto por:

- La interfaz **Observador**.
- La clase **Sujeto**.
- La clase concreta **Usuario**.
- Una colección de observadores dentro de **Sujeto**.

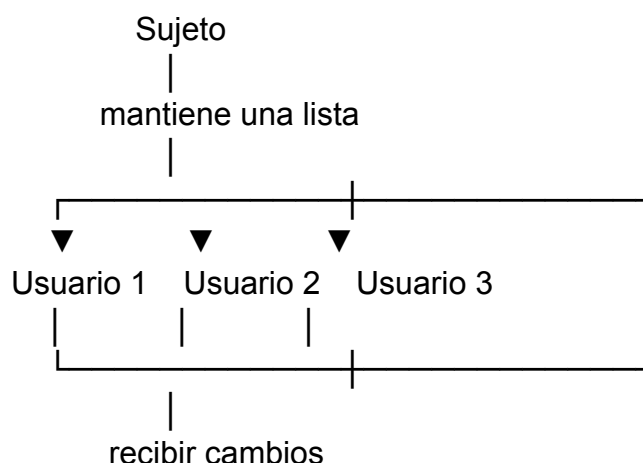
La interfaz **Observador** define el método **actualizar()**, que debe ser implementado por cualquier objeto que quiera recibir notificaciones.

La clase **Sujeto** mantiene una lista de observadores registrados. Para administrar esta lista dispone de métodos que permiten agregar y eliminar observadores. Cuando ocurre un cambio, el sujeto utiliza el método **notificarObservadores()** para informar a todos los observadores registrados.

El funcionamiento puede resumirse de la siguiente manera:

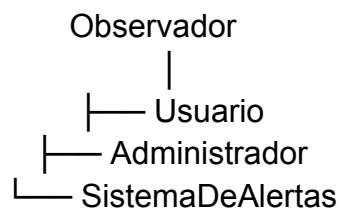
1. Los objetos interesados implementan la interfaz **Observador**.
2. Los observadores se registran en el **Sujeto**.
3. El **Sujeto** mantiene una lista de los observadores registrados.
4. Ocurre un cambio o evento en el **Sujeto**.
5. El **Sujeto** recorre la lista de observadores.
6. Cada observador recibe una notificación mediante el método **actualizar()**.

Conceptualmente:



Una característica importante es que el **Sujeto** trabaja con la interfaz **Observador** y no necesita conocer los detalles de cada observador concreto.

Por ejemplo, podría tener:



Todos pueden recibir la misma notificación aunque tengan comportamientos diferentes.

De esta manera, Observer reduce el acoplamiento y permite incorporar nuevos observadores sin modificar la estructura principal del sujeto.

3. Consecuencias

La utilización del patrón Observer presenta diferentes ventajas, desventajas e implicancias que deben considerarse.

- **Ventajas:**

Reduce el acoplamiento

El sujeto no necesita conocer los detalles de los objetos que reciben las notificaciones. Solamente necesita trabajar con la interfaz **Observador**.

Permite agregar nuevos observadores

Es posible incorporar nuevos tipos de observadores sin modificar la clase **Sujeto**, siempre que implementen la interfaz correspondiente.

Facilita la comunicación entre objetos

Permite que varios objetos reaccionen ante un mismo evento de manera automática.

Favorece la reutilización

La misma estructura puede utilizarse para diferentes tipos de eventos y sistemas de notificaciones.

Permite administrar suscripciones

Los observadores pueden registrarse o dejar de recibir notificaciones mediante operaciones como `agregarObservador()` y `eliminarObservador()`.

- **Desventajas:**

Puede generar muchas notificaciones

Si existe una gran cantidad de observadores, un cambio en el sujeto puede provocar muchas llamadas a `actualizar()`, lo que podría afectar el rendimiento.

El flujo de ejecución puede ser más difícil de seguir

Cuando ocurre un evento, la ejecución puede propagarse hacia varios objetos. Esto puede dificultar identificar rápidamente qué componentes se ejecutan como consecuencia de una determinada acción.

Existe dependencia entre el evento y los observadores

Aunque se reduce el acoplamiento entre las clases, los observadores siguen dependiendo de las notificaciones que genera el sujeto.

Puede provocar problemas de memoria

Si los observadores dejan de utilizarse pero continúan registrados en el sujeto, pueden permanecer en memoria innecesariamente. Por esta razón, es importante gestionar correctamente las suscripciones y eliminaciones.

- **Implicancias:**

Observer resulta especialmente útil cuando varios objetos necesitan reaccionar ante cambios producidos por otro objeto y se desea evitar una dependencia directa entre ellos.

La principal implicancia del patrón es que transforma la comunicación directa entre objetos en un mecanismo basado en **suscripciones y notificaciones**.

Esto proporciona flexibilidad, pero también significa que el comportamiento del sistema puede estar distribuido entre varios objetos. Por esta razón, cuando se utiliza Observer se debe prestar atención a la cantidad de observadores, la frecuencia de las notificaciones y la correcta gestión de las suscripciones.

En conclusión, Observer es una alternativa adecuada para sistemas basados en eventos, notificaciones y actualizaciones automáticas, especialmente cuando existe una relación de uno a muchos entre el objeto que produce un cambio y los objetos interesados en recibir información sobre él.